RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)
Department of Artificial Intelligence and Data Science
V Semester B. Tech AIDS

AD23532 - Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Ex 5: Information Extraction using spaCy

Register Number: 241801177 **Date:** 09-10-2026

Dataset: Amazon Fine Food Reviews (Kaggle) **Tools:** Python, Pandas, spaCy (en_core_web_sm), Matplotlib

Aim

To process customer reviews with the spaCy NLP library and build an information extraction system that identifies tokens, lemmas, part-of-speech tags and named entities (PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT) and organises them into a structured format for customer feedback analysis.

Task 1: Load the dataset, install and import the required libraries

In [1]:

```
# Install (run once):  pip install spacy pandas matplotlib
#                      python -m spacy download en_core_web_sm
import textwrap
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
import spacy

pd.set_option('display.max_columns', None); pd.set_option('display.width', 200)
df = pd.read_csv('Reviews.csv')
print("Dataset loaded successfully")
print("Shape:", df.shape)
nlp = spacy.load('en_core_web_sm', disable=['parser'])   # parser is not needed for POS / lemma / NER
print("spaCy", spacy.__version__, "| model:", nlp.meta['name'], nlp.meta['version'])
```

Out [1]:

```
Dataset loaded successfully
Shape: (568454, 10)
spaCy 3.8.16 | model: core_web_sm 3.8.0
```

> **Observation:** The Amazon Fine Food Reviews file has 568,454 reviews and 10 columns. Only the Text column (the review text) is used. The spaCy model en_core_web_sm (version 3.8.0) is a small English pipeline trained on web text, so it runs quickly on a CPU.

Task 2: Clean the dataset

In [2]:

```
data = df[['Text']].copy()                                        # select the Text column
print("Missing:", data['Text'].isnull().sum(), "| Duplicates:", data.duplicated().sum())
data = data.dropna(subset=['Text']).drop_duplicates(subset='Text') # remove nulls and duplicates
data = data.head(10000).reset_index(drop=True)                    # first 10,000 reviews
print("Reviews retained:", len(data))
```

Out [2]:

```
Missing: 0 | Duplicates: 174875
Reviews retained: 10000
```

> **Observation:** No review has a missing Text value, but 174,875 reviews are exact duplicates and were removed before the first 10,000 unique reviews were kept. The same 10,000 reviews were used in Ex 4.

Task 3: Apply the spaCy pipeline - tokens, lemmas and POS tags of five reviews

In [3]:

```
docs = list(nlp.pipe(data['Text'], batch_size=64))   # one pass over all 10,000 reviews
for i in range(5):
    d = docs[i]
    print(f"\nReview {i}")
    print(textwrap.fill("Original: " + d.text[:250] + " ...", 105, subsequent_indent='          '))
    table = pd.DataFrame({'Token': [t.text for t in d[:12]], 'Lemma': [t.lemma_ for t in d[:12]],
                          'POS': [t.pos_ for t in d[:12]], 'Tag': [t.tag_ for t in d[:12]]})
    print(table.T.to_string(header=False))          # first 12 tokens
```

Out [3]:

```
Review 0
Original: I have bought several of the Vitality canned dog food products and have found them all to be of
          good quality. The product looks more like a stew than a processed meat and it smells better. My
          Labrador is finicky and she appreciates this product bette ...
Token     I  have  bought  several   of  the  Vitality  canned   dog  food  products    and
Lemma     I  have     buy  several   of  the  Vitality     can   dog  food   product    and
POS    PRON   AUX    VERB      ADJ  ADP  DET     PROPN    VERB  NOUN  NOUN      NOUN  CCONJ
Tag     PRP   VBP     VBN       JJ   IN   DT       NNP     VBN    NN    NN       NNS     CC

Review 1
Original: Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized
          unsalted. Not sure if this was an error or if the vendor intended to represent the product as
          "Jumbo". ...
Token  Product  arrived  labeled   as  Jumbo  Salted  Peanuts    ...  the  peanuts  were  actually
Lemma  product   arrive    label   as  Jumbo  Salted  Peanuts    ...  the   peanut    be  actually
POS       NOUN     VERB     VERB  ADP  PROPN   PROPN    PROPN  PUNCT  DET     NOUN   AUX       ADV
Tag         NN      VBD      VBN   IN    NNP     NNP      NNP      :   DT      NNS   VBD        RB

Review 2
Original: This is a confection that has been around a few centuries.  It is a light, pillowy citrus
          gelatin with nuts - in this case Filberts. And it is cut into tiny squares and then liberally
          coated with powdered sugar.  And it is a tiny mouthful of heaven.  ...
Token  This   is    a  confection  that   has  been  around    a  few  centuries      .
Lemma  this   be    a  confection  that  have    be  around    a  few    century      .
POS     DET  AUX  DET        NOUN  PRON   AUX   AUX     ADP  DET  ADJ       NOUN  PUNCT
Tag      DT  VBZ   DT          NN   WDT   VBZ   VBN      IN   DT   JJ        NNS      .

Review 3
Original: If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got
          this in addition to the Root Beer Extract I ordered (which was good) and made some cherry soda.
          The flavor is very medicinal. ...
Token     If   you  are  looking  for  the  secret  ingredient   in  Robitussin     I  believe
Lemma     if   you   be     look  for  the  secret  ingredient   in  Robitussin     I  believe
POS    SCONJ  PRON  AUX     VERB  ADP  DET     ADJ        NOUN  ADP       PROPN  PRON     VERB
Tag       IN   PRP  VBP      VBG   IN   DT      JJ          NN   IN         NNP   PRP      VBP

Review 4
Original: Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very
          quick.  If your a taffy lover, this is a deal. ...
Token  Great  taffy   at    a  great  price      .         There  was    a  wide
Lemma  great  taffy   at    a  great  price      .         there   be    a  wide
POS      ADJ   NOUN  ADP  DET    ADJ   NOUN  PUNCT  SPACE   PRON  AUX  DET   ADJ
Tag       JJ     NN   IN   DT     JJ     NN      .    _SP     EX  VBD   DT    JJ
```

> **Observation:** The pipeline splits each review into tokens and gives each token a lemma (base form) and a part-of-speech tag. Lemmatization maps "bought" to "buy", "products" to "product", "centuries" to "century", and "was" and "were" to "be". In Review 0, "Vitality" is tagged PROPN (proper noun) and "canned" is tagged VERB, although it acts as an adjective in "canned dog food", so the small model is not always exact. Review 4 shows a SPACE token created by the double space after a full stop.

Task 4: POS categories - Nouns, Verbs, Adjectives and Adverbs

In [4]:

```
groups = {'NOUN': Counter(), 'VERB': Counter(), 'ADJ': Counter(), 'ADV': Counter()}
for d in docs:
    for t in d:
        if t.pos_ in groups and t.is_alpha and not t.is_stop:
            groups[t.pos_][t.lemma_.lower()] += 1
for pos, c in groups.items():
    print(f"\nTop 10 {pos}s:", ", ".join(f"{w} ({n})" for w, n in c.most_common(10)))
```

Out [4]:

```
Top 10 NOUNs: product (3292), flavor (3184), coffee (2934), taste (2092), food (1999), time (1679), tea (1673),
    cup (1576), bag (1531), price (1370)

Top 10 VERBs: try (2753), love (2716), buy (2614), find (2306), like (2231), taste (2188), eat (1792), use
    (1412), think (1325), order (1276)

Top 10 ADJs: good (4532), great (2793), little (1368), well (975), hot (866), sweet (857), free (792), small
    (731), nice (692), easy (685)
```

```
Top 10 ADVs: definitely (506), far (474), well (470), pretty (452), actually (409), highly (381), usually (371),
    long (360), probably (347), away (327)
```

> **Observation:** The most common content words match the food-review domain. Nouns: product, flavor, coffee, taste, food. Verbs: try, love, buy, find, like. Adjectives: good (4,532), great (2,793), little, sweet. Adverbs: definitely, far, well, pretty. Stopwords were excluded here. Some words appear in two categories ("taste" as noun and verb, "well" as adjective and adverb), because the tag depends on the sentence.

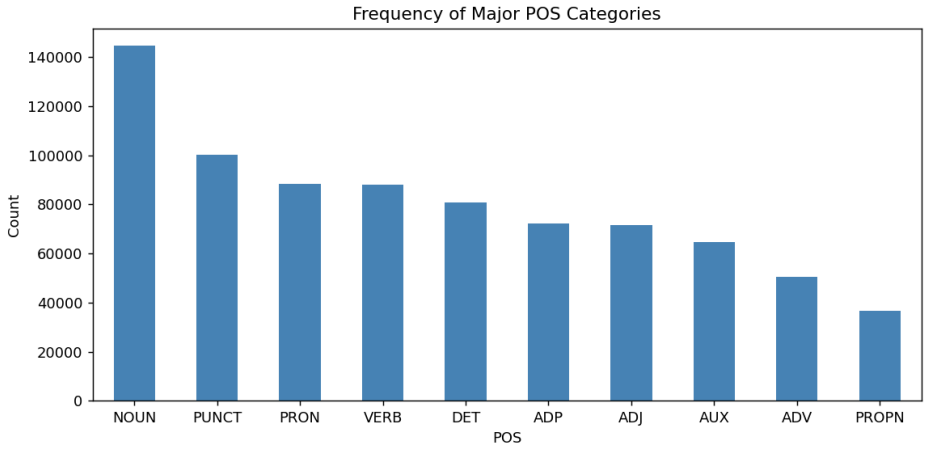
Task 5: Frequency of the major POS categories

In [5]:

```
pos_counts = Counter(t.pos_ for d in docs for t in d)
pos_df = pd.DataFrame(pos_counts.most_common(), columns=['POS', 'Count'])
pos_df['Percent'] = (pos_df['Count'] / pos_df['Count'].sum() * 100).round(2)
print(pos_df.to_string(index=False))
pos_df.head(10).set_index('POS')['Count'].plot(kind='bar', figsize=(9, 4.5), color='steelblue', rot=0)
plt.title('Frequency of Major POS Categories'); plt.ylabel('Count'); plt.tight_layout(); plt.show()
```

Out [5]:

```
   POS  Count  Percent
  NOUN 144497    15.97
 PUNCT 100112    11.06
  PRON  88240     9.75
  VERB  87917     9.72
   DET  80875     8.94
   ADP  72211     7.98
   ADJ  71483     7.90
   AUX  64765     7.16
   ADV  50619     5.59
 PROPN  36754     4.06
 CCONJ  32640     3.61
  PART  24982     2.76
 SPACE  16881     1.87
 SCONJ  13630     1.51
   NUM  11798     1.30
     X   2976     0.33
   SYM   2819     0.31
  INTJ   1736     0.19
```



> **Observation:** The 10,000 reviews contain 904,935 tokens. Nouns are the most frequent category (15.97%), followed by punctuation (11.06%), pronouns (9.75%), verbs (9.72%) and determiners (8.94%). Adjectives (7.90%) and adverbs (5.59%) are also common, which fits opinion-rich review text.

Task 6: Named Entity Recognition (PERSON, ORG, DATE, MONEY, QUANTITY, PRODUCT)

In [6]:

```
wanted = ['PERSON', 'ORG', 'DATE', 'MONEY', 'QUANTITY', 'PRODUCT']
rows = [(i, e.text, e.label_) for i, d in enumerate(docs) for e in d.ents if e.label_ in wanted]
ent_df = pd.DataFrame(rows, columns=['Review_ID', 'Entity', 'Label'])   # structured DataFrame
print("Entities found:", len(ent_df))
print(ent_df['Label'].value_counts().to_string())
print(ent_df.head(15).to_string(index=False))
print("\nTop 5 entities per label:")
for lab in wanted:
    sub = ent_df[ent_df['Label'] == lab]['Entity'].str.lower().value_counts().head(5)
    print(f"  {lab:9s}:", ", ".join(f"{k} ({v})" for k, v in sub.items()) or "none found")
```

Out [6]:

```
Entities found: 16014
Label
ORG         6702
PERSON      4414
DATE        3056
MONEY        896
QUANTITY     671
PRODUCT      275
 Review_ID                            Entity    Label
         0                           Vitality      ORG
         0                       My Labrador   PERSON
         1                       Jumbo Salted      ORG
         2            around a few centuries     DATE
         2            pillowy citrus gelatin      ORG
         2                        C.S. Lewis'      ORG
         2                             Edmund  PRODUCT
         2                           Sisters   PERSON
         2                             Witch      ORG
         4                          Delivery   PERSON
         5                       five pound QUANTITY
         5                    only two weeks     DATE
         6                       Fralinger's      ORG
         8                        Wheatgrass   PERSON
        10 Tequila Picante Gourmet de Inclan      ORG

Top 5 entities per label:
  PERSON   : keurig (209), switch (127), bisquick (101), newman (78), wolfgang puck (61)
  ORG      : amazon (1072), amazon.com (90), newman (65), starbucks (59), bbq (55)
  DATE     : years (108), christmas (107), today (104), daily (84), the day (72)
  MONEY    : 1 (25), 10 (20), 20 (17), 5 (17), 2 (17)
  QUANTITY : a ton (20), 34 grams (19), 1 cup (14), 8.3 ounces (12), 12 ounce (11)
  PRODUCT  : gatorade (18), subscribe (15), starbucks (8), ca (7), trident (5)
```

Observation: The model found 16,014 entities in the six requested labels: ORG 6,702, PERSON 4,414, DATE 3,056, MONEY 896, QUANTITY 671 and PRODUCT 275. Amazon is the top organisation (1,072 mentions), and DATE and QUANTITY entities such as "five pound" and "34 grams" are mostly correct. However, the small model makes many mistakes on food brands: Keurig, Switch and Bisquick are mostly labelled PERSON (437 of 454 mentions), "My Labrador" and "Delivery" are labelled PERSON, and "pillowy citrus gelatin" is labelled ORG. PRODUCT is rare (275) and includes noise such as "subscribe". MONEY entries are often bare numbers ("1", "10", "20") taken from prices. A larger model or domain training would improve this.

Task 7: Information extraction function

In [7]:

```
def extract_info(review):
    d = nlp(review)
    return {'Tokens': [t.text for t in d], 'Lemmas': [t.lemma_ for t in d],
            'POS_Tags': [t.pos_ for t in d],
            'Named_Entities': [e.text for e in d.ents], 'Entity_Labels': [e.label_ for e in d.ents]}

print("Function extract_info(review) defined")
```

Out [7]:

```
Function extract_info(review) defined
```

Observation: extract_info() runs the pipeline on one review and returns five lists: tokens, lemmas, POS tags, named entities and entity labels. They have the same order as the text, so they can be placed in a DataFrame or saved for later analysis.

Task 8: Test the function on three new customer reviews

In [8]:

```
new_reviews = [
    "I ordered the Starbucks coffee pods from Amazon in March and paid $25 for a box of 24 pods. John loved the taste.",
    "The Nestle chocolate arrived late and the packaging was damaged, very disappointed with this seller.",
    "Bought two 500 gram bags of Lipton green tea last week for 12 dollars, great flavor and fast delivery.",
]
results = [extract_info(r) for r in new_reviews]
print("Reviews processed:", len(results))
for n, info in enumerate(results, 1):
    print(f"Review {n}: {len(info['Tokens'])} tokens, {len(info['Named_Entities'])} entities")
```

Out [8]:

```
Reviews processed: 3
Review 1: 26 tokens, 6 entities
Review 2: 17 tokens, 1 entities
Review 3: 21 tokens, 4 entities
```

Observation: The function worked on all three new reviews, returning 26, 17 and 21 tokens and 6, 1 and 4 entities.

Task 9: Display the extracted information in a structured format

In [9]:

```
for n, (review, info) in enumerate(zip(new_reviews, results), 1):
    print(f"Review {n}: {review}")
    trip = [f"{t}/{l}/{p}" for t, l, p in zip(info['Tokens'], info['Lemmas'], info['POS_Tags'])]
    print(textwrap.fill("Token/Lemma/POS: " + " ".join(trip), 105, subsequent_indent='                  '))
    print(pd.DataFrame({'Entity': info['Named_Entities'], 'Label': info['Entity_Labels']}).to_string(index=False))
    print()
```

Out [9]:

```
Review 1: I ordered the Starbucks coffee pods from Amazon in March and paid $25 for a box of 24 pods. John loved the taste.
Token/Lemma/POS: I/I/PRON ordered/order/VERB the/the/DET Starbucks/Starbucks/PROPN coffee/coffee/NOUN
                 pods/pod/NOUN from/from/ADP Amazon/Amazon/PROPN in/in/ADP March/March/PROPN
                 and/and/CCONJ paid/pay/VERB $/$/SYM 25/25/NUM for/for/ADP a/a/DET box/box/NOUN of/of/ADP
                 24/24/NUM pods/pod/NOUN ././PUNCT John/John/PROPN loved/love/VERB the/the/DET
                 taste/taste/NOUN ././PUNCT
   Entity    Label
Starbucks      ORG
   Amazon      ORG
    March     DATE
       25    MONEY
       24 CARDINAL
     John   PERSON

Review 2: The Nestle chocolate arrived late and the packaging was damaged, very disappointed with this seller.
Token/Lemma/POS: The/the/DET Nestle/Nestle/PROPN chocolate/chocolate/NOUN arrived/arrive/VERB
                 late/late/ADV and/and/CCONJ the/the/DET packaging/packaging/NOUN was/be/AUX
                 damaged/damage/VERB ,/,/PUNCT very/very/ADV disappointed/disappointed/ADJ with/with/ADP
                 this/this/DET seller/seller/NOUN ././PUNCT
Entity  Label
Nestle PERSON

Review 3: Bought two 500 gram bags of Lipton green tea last week for 12 dollars, great flavor and fast delivery.
Token/Lemma/POS: Bought/buy/VERB two/two/NUM 500/500/NUM gram/gram/NOUN bags/bag/NOUN of/of/ADP
                 Lipton/Lipton/PROPN green/green/ADJ tea/tea/NOUN last/last/ADJ week/week/NOUN
                 for/for/ADP 12/12/NUM dollars/dollar/NOUN ,/,/PUNCT great/great/ADJ flavor/flavor/NOUN
                 and/and/CCONJ fast/fast/ADJ delivery/delivery/NOUN ././PUNCT
      Entity    Label
two 500 gram QUANTITY
      Lipton      ORG
   last week     DATE
  12 dollars    MONEY
```

Observation: Review 1 is handled well: Starbucks and Amazon are ORG, March is DATE, 25 is MONEY and John is PERSON. "24" is labelled CARDINAL, a label outside the six requested ones, which the function still returns. In Review 2 the model labels the brand Nestle as PERSON instead of ORG, the same kind of brand error seen in Task 6. Review 3 is extracted correctly: "two 500 gram" is QUANTITY, Lipton is ORG, "last week" is DATE and "12 dollars" is MONEY, although the two numbers are merged into one quantity. Lemmas are also correct (ordered to order, paid to pay, loved to love, bags to bag).

Summary table of the extracted entities

In [10]:

```
summary = pd.DataFrame({'Review': [f"Review {n}" for n in range(1, 4)],
                        'Entities (Label)': [", ".join(f"{e} ({l})" for e, l in zip(i['Named_Entities'], i['Entity_Labels']))
                                             for i in results]})
print(summary.to_string(index=False))
```

Out [10]:

```
  Review                                                                  Entities (Label)
Review 1 Starbucks (ORG), Amazon (ORG), March (DATE), 25 (MONEY), 24 (CARDINAL), John (PERSON)
Review 2                                                                   Nestle (PERSON)
Review 3           two 500 gram (QUANTITY), Lipton (ORG), last week (DATE), 12 dollars (MONEY)
```

Result

Thus an information extraction system was built with spaCy on the first 10,000 unique reviews of the Amazon Fine Food Reviews dataset (904,935 tokens). The pipeline produced tokens, lemmas and part-of-speech tags, and the most frequent nouns, verbs, adjectives and adverbs were identified (nouns 15.97% of all tokens). Named entity recognition found 16,014 entities across the six requested labels, led by ORG (6,702) and PERSON (4,414), though the small model often mislabels food brands as PERSON and rarely recognises PRODUCT. An extract_info() function returning tokens, lemmas, POS tags, entities and entity labels was tested on three new reviews, and its output was displayed in a structured format.